AprilTag Tracking

1. Function Description

After the program starts, when the program detects the AprilTag, it will track the AprilTag so that the center of the AprilTag coincides with the center of the image.

2. Startup and Operation

2.1 Startup Command

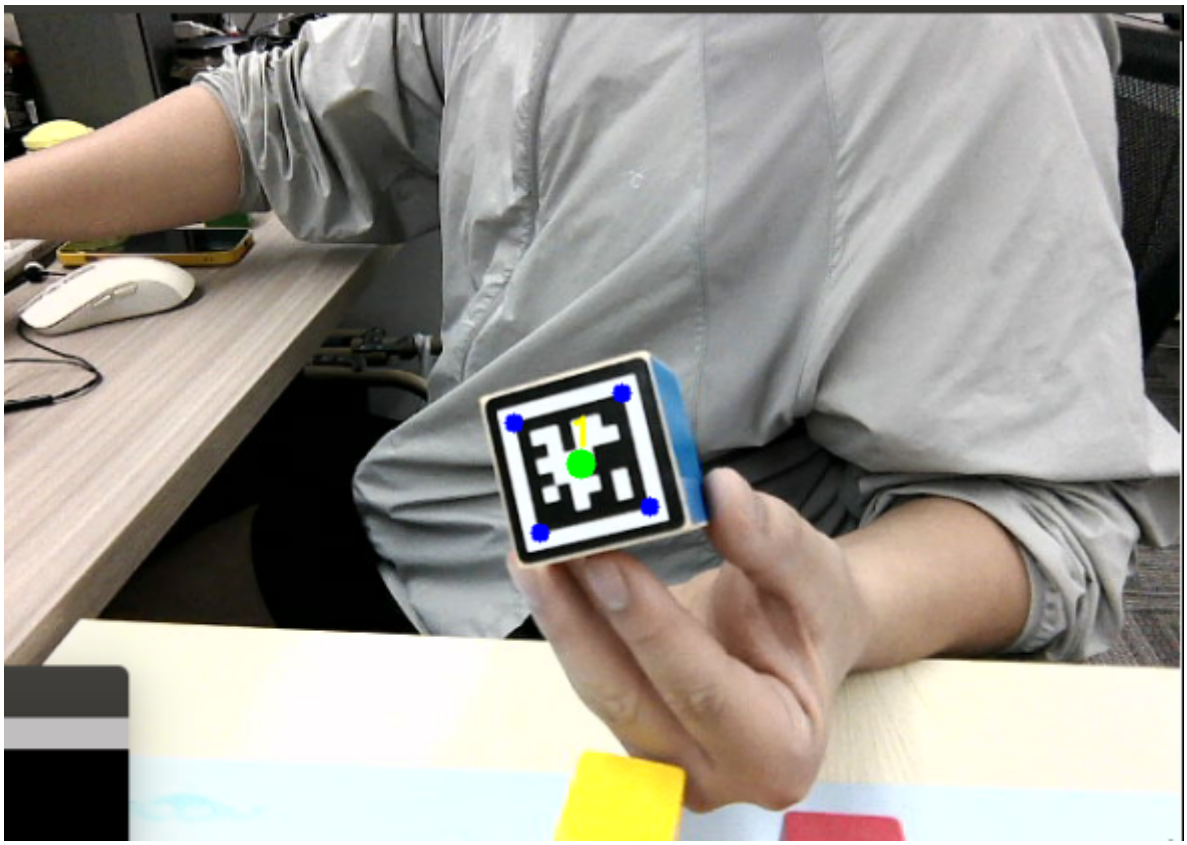
Enter the following commands in the terminal to start:

```
# Start camera and inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start robotic arm control program
ros2 run dofbot_sorting_3d grasp
# Start AprilTag tracking program
ros2 run dofbot_follow apriltag_follow_2D
```

2.2 Operation

After the program starts, hold a 3cm*3cm AprilTag wooden block in the image, and the robotic arm will adjust its pose, slowly moving the AprilTag. The robotic arm follows the AprilTag's movement and continuously adjusts its own posture.

Note: When tracking AprilTag, move slowly within the camera's field of view



3. Core Code Analysis

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_follow/dofbot_follow/apriltag_follow_2D.py
```

Import necessary libraries:

```
import cv2
import os
import numpy as np
from sensor_msgs.msg import Image, CameraInfo
import message_filters
from dofbot_sorting_3d.vutils import draw_tags
from dt_apriltags import Detector
from cv_bridge import CvBridge
import cv2 as cv

from std_msgs.msg import Float32,Bool,Int16,UInt16,String
encoding = ['16UC1', '32FC1']
import time
import math
from rclpy.node import Node
import rclpy
from sensor_msgs.msg import Image
import threading
import yaml
import dofbot_follow.PID as PID
from Arm_Lib import Arm_Device
import os
```

Initialize program parameters, create publishers and subscribers:

```
class AprilTagTrackNode(Node):
    # Class for tracking AprilTags
    # 机械臂跟踪AprilTag类

    def __init__(self, name):
        # Initialize the ROS2 node with the given name
        # 使用给定的名称初始化ROS2节点
        super().__init__(name)

        # Create an instance of the robotic arm device class
        # 创建机械臂设备类的实例
        self.Arm = Arm_Device()

        # Define initial joint angles for the robot arm
        # 定义机械臂的初始关节角度
        self.init_joints = [90, 150, 0, 20, 90, 0]

        # Move the arm to the initial position
        # 将机械臂移动到初始位置
        self.Arm.Arm_serial_servo_write6_array(self.init_joints, 2000)

        # Bridge for converting between OpenCV images and ROS2 Image messages
        # 用于在OpenCV图像和ROS2 Image消息之间转换的桥接器
```

```
        self.rgb_bridge = CvBridge()

        # Previous time record for calculating FPS
        # 用于计算FPS的先前时间记录
        self.pr_time = time.time()

        # Initialize the AprilTag detector with specific parameters
        # 使用特定参数初始化AprilTag检测器
        self.at_detector = Detector(searchpath=['apriltags'],
                                    families='tag36h11',
                                    nthreads=8,
                                    quad_decimate=2.0,
                                    quad_sigma=0.0,
                                    refine_edges=1,
                                    decode_sharpening=0.25,
                                    debug=0)

        # Create a subscription to receive raw camera images
        # 创建订阅以接收原始相机图像
        self.subscription = self.create_subscription(
            Image, '/image_raw', self.ImageCallback, 1)

        # Variables to store the x and y position of the detected tag
        # 存储检测到的标签x和y位置的变量
        self.px = 0.0
        self.py = 0.0

        # Target servo positions for x and y axes
        # x轴和y轴的目标舵机位置
        self.target_servox = 90
        self.target_servoy = 45

        # PID controllers for x and y axis servo control
        # 用于x轴和y轴舵机控制的PID控制器
        self.xservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)
        self.yservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)

        # Flags indicating if servo output is out of range
        # 标志位，表示舵机输出是否超出范围
        self.y_out_range = False
        self.x_out_range = False

        # Variables to store calculated position values
        # 存储计算位置值的变量
        self.a = 0
        self.b = 0
```

ImageCallback callback function:

```
def ImageCallback(self, color_frame):
    # Convert ROS2 Image message to OpenCV RGB image format
    # 将ROS2 Image消息转换为OpenCV RGB图像格式
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')

    # Convert RGB to Grayscale for detection
    # 将RGB转换为灰度图以进行检测
```

```
    tags = self.at_detector.detect(cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY),
False, None, 0.025)

    # Sort detected tags by their ID in ascending order
    # 按标签ID升序排列检测到的标签
    tags = sorted(tags, key=lambda tag: tag.tag_id)

    # Draw bounding boxes and centers on the image
    # 在图像上绘制边界框和中心点
    # Corners in red, center in green
    # 角点为红色，中心点为绿色
    draw_tags(rgb_image, tags, corners_color=(0, 0, 255), center_color=(0, 255,
0))

    # Convert image to BGR format for display
    # 将图像转换为BGR格式以便显示
    frame1 = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)

    # Display the result image in a window
    # 在窗口中显示结果图像
    cv2.imshow("result_image", frame1)

    # Check for key press (required for imshow to work properly)
    # 检查按键（imshow正常工作必需的）
    key = cv2.waitKey(1)

    # Process only if at least one tag is detected
    # 仅当检测到至少一个标签时进行处理
    if len(tags) > 0:
        # Get the ID of the first detected tag
        # 获取第一个检测到的标签的ID
        cur_id = tags[0].tag_id

        # Get the center coordinates of the tag
        # 获取标签的中心坐标
        center_x, center_y = tags[0].center

        # Check if the tag center deviates significantly from image center
        # 检查标签中心是否明显偏离图像中心
        # Image center: (320, 240) for 640x480 resolution
        # 图像中心：（320, 240），对应640x480分辨率
        # Threshold: 10 pixels
        # 阈值：10像素
        if (abs(center_x - 320) > 10 or abs(center_y - 240) > 10):
            # Call XY tracking function to adjust arm position
            # 调用XY跟踪函数以调整机械臂位置
            self.XY_track(center_x, center_y)

            # Print tracking status
            # 打印跟踪状态
            print("Tracking")
            print("-------------------------------------")
```

XY_track function:

```
def XY_track(self, center_x, center_y):
    # Store the current detected tag center position
```

```
    # 存储当前检测到的标签中心位置
    self.px = center_x
    self.py = center_y

    # X-axis servo control logic
    # X轴舵机控制逻辑
    # Check boundary conditions to prevent excessive movement
    # 检查边界条件以防止过度移动
    if not (self.target_servox >= 180 and center_x <= 320 and self.a == 1 or
            self.target_servox <= 0 and center_x >= 320 and self.a == 1):
        # Normal tracking mode (not in extreme condition)
        # 正常跟踪模式
        if self.a == 0:
            # Set the current tag x-position as the PID system output
            # 将当前标签x位置设为PID系统输出
            self.xservo_pid.SystemOutput = center_x

            # Handle out-of-range situations
            # 处理超出范围的情况
            if self.x_out_range == True:
                if self.target_servox < 0:
                    self.target_servox = 0
                    self.xservo_pid.SetStepSignal(630)
                if self.target_servox > 0:
                    self.target_servox = 180
                    self.xservo_pid.SetStepSignal(10)
                self.x_out_range = False
            else:
                # Normal operation: set step signal to image center x (320)
                # 正常操作：将目标值设为图像中心x坐标（320）
                self.xservo_pid.SetStepSignal(320)
                self.x_out_range = False

            # Set PID inertia time parameters
            # 设置PID惯性时间参数
            self.xservo_pid.SetInertiaTime(0.01, 0.1)

            # Calculate target servo value
            # 计算目标舵机值
            target_valuex = int(1500 + self.xservo_pid.SystemOutput)

            # Convert to servo angle
            # 转换为舵机角度
            self.target_servox = int((target_valuex - 500) / 10) - 10

            # Check if servo angle exceeds limits
            # 检查舵机角度是否超出限制
            if self.target_servox > 180:
                self.x_out_range = True
            if self.target_servox < 0:
                self.x_out_range = True
            print("self.target_servox: ", self.target_servox)

    # Y-axis servo control logic
    # Y轴舵机控制逻辑
    if not (self.target_servoy >= 180 and center_y <= 240 and self.b == 1 or
            self.target_servoy <= 0 and center_y >= 240 and self.b == 1):
        if self.b == 0:
```

```
            # Set the current tag y-position as the PID system output
            # 将当前标签y位置设为PID系统输出
            self.yservo_pid.SystemOutput = center_y

            # Handle out-of-range situations
            # 处理超出范围的情况
            if self.y_out_range == True:
                self.yservo_pid.SetStepSignal(450)
                self.y_out_range = False
            else:
                # Normal operation: set step signal to image center y (240)
                # 正常操作：目标信号设为图像中心y坐标（240）
                self.yservo_pid.SetStepSignal(240)

            # Set PID inertia time parameters
            # 设置PID惯性时间参数
            self.yservo_pid.SetInertiaTime(0.01, 0.1)

            # Calculate target servo value
            # 计算目标舵机值
            target_valuey = int(1500 + self.yservo_pid.SystemOutput)

            # Enforce minimum boundary
            # 强制执行最小边界
            if target_valuey <= 1000:
                target_valuey = 1000
                self.y_out_range = True

            # Convert to servo angle with offset
            # 转换舵机角度
            self.target_servoy = int((target_valuey - 500) / 10) - 55

            # Clamp servo angle to valid range
            # 将舵机角度限制在有效范围内
            if self.target_servoy > 180:
                self.target_servoy = 180
            if self.target_servoy < 0:
                self.target_servoy = 0

            # Calculate robotic arm joints
            # 计算机械臂关节
            joint2 = 120 + self.target_servoy
            joint3 = self.target_servoy / 4.5
            joint4 = self.target_servoy / 3


    # Joint angles array and send to arm
    # 关节角度数组并发送给机械臂
    joints_0 = [float(self.target_servox / 1), float(joint2), float(joint3),
float(joint4), 90.0, 30.0]
    # Send joint commands to the robotic arm
    # 向机械臂发送关节控制命令
    self.Arm.Arm_serial_servo_write6_array(joints_0, 2000)
```